

LAPORAN PRAKTIKUM IOT INTERNET LAYER

Pengujian IPv4, IPv6, Static Routing, Forwarding, ARP, Neighbor Discovery, dan Path MTU Discovery Menggunakan Linux Network Namespaces.

<https://github.com/ridlofw/internet-layer>

Nama Mahasiswa : Ridlo Fanata Wicaksana
NIM : 26/591572/NPA/20034
Nama Mahasiswa : Javier Gavra Abhinaya
NIM : 26/592020/NPA/20050
Nama Mahasiswa : Nur Alif Maulana Syafrudin
NIM : 26/591608/NPA/20042
Mata Kuliah : Internet of Things

RINGKASAN PROYEK

Praktikum Lapisan Internet (Network Layer) ini bertujuan untuk menganalisis dan memverifikasi mekanisme pengalamatan dual-stack IPv4 dan IPv6, penerusan paket (forwarding), rute statis dua arah, resolusi alamat link-layer (ARP dan Neighbor Discovery), penanganan batasan Hop Limit/TTL, serta dampak batasan Path MTU Discovery (PMTU) dalam arsitektur jaringan Internet of Things (IoT). Seluruh eksperimen dilaksanakan dalam lingkungan terisolasi Linux Network Namespaces di atas 1 Virtual Machine Ubuntu 26.04 LTS tanpa memerlukan perangkat keras fisik.

Capaian Pembelajaran

1. **Isolasi Topologi Terisolasi:** Membangun dan mengonfigurasi topologi 3 node terisolasi (iot-sensor, iot-router, iot-cloud) menggunakan Virtual Ethernet Pairs (veth).
2. **Pengalamatan Dual-Stack & Routing:** Mengonfigurasi alamat IPv4 (/24) dan IPv6 (/64) serta tabel rute statis untuk menjamin konektivitas dua arah.
3. **Resolusi Alamat Link-Layer:** Menganalisis perbedaan operasional antara Address Resolution Protocol (ARP) pada IPv4 dan Neighbor Discovery Protocol (NDP ICMPv6 Type 135/136) pada IPv6.
4. **Diagnosis Kegagalan Jaringan:** Mendiagnosis dampak habisnya Hop Limit (TTL=1), hilangnya rute balik (missing return route), dan matinya fungsi IP forwarding pada router.
5. **Path MTU Discovery (PMTU):** Mengamati respon ICMPv6 Packet Too Big (Type 2 Code 0) akibat pembatasan MTU link 1280 bytes dan menganalisis fragmentasi paket.
6. **Layanan Aplikasi IPv6:** Menguji pengiriman data aplikasi menggunakan HTTP Collector Server dan Klien Socket TCP di atas IPv6.

BAB 1: PENDAHULUAN, KONSEP DASAR, DAN LINGKUNGAN KERJA

Lapisan Internet (Network Layer) bertanggung jawab untuk menyediakan pengalamatan logis logis dan penerusan paket data melintasi beberapa jaringan yang heterogen. Dalam ekosistem IoT, node sensor nirkabel berdaya rendah sering kali terhubung ke jaringan lokal sebelum diteruskan melalui gateway/router menuju sistem pengumpul data cloud.

Untuk mensimulasikan arsitektur ini secara realistis di dalam 1 sistem operasi VM Ubuntu 26.04 LTS, digunakan fitur Linux Network Namespaces. Fitur ini menyediakan stack jaringan independen (tabel rute, antarmuka, dan cache neighbor sendiri) untuk setiap node virtual.

1.1 Topologi & Alokasi Alamat IP Jaringan

Topologi praktikum terdiri dari 3 network namespace yang terhubung dalam skema dua subnet terpisah melalui perangkat router virtual:

Namespace	Antarmuka	Alamat IPv4	Alamat IPv6 (/64)	Peran Perangkat
iot-sensor	s0	10.10.1.2 /24	2001:db8:1::2 /64	Node Sensor (Pengirim)
iot-router	r0	10.10.1.1 /24	2001:db8:1::1 /64	Gateway sisi Sensor
iot-router	r1	10.10.2.1 /24	2001:db8:2::1 /64	Gateway sisi Cloud
iot-cloud	c0	10.10.2.2 /24	2001:db8:2::2 /64	Cloud Collector (Tujuan)

1.2 Verifikasi Lingkungan Kerja VM Ubuntu

Sebelum memulai simulasi, dilakukan verifikasi paket jaringan utama seperti iproute2, iputils-ping, tcpdump, dan python3 pada sistem Ubuntu 26.04 LTS.

```
ridlo@ubuntu:~$ ip -V && tcpdump --version && python3 --version
ip utility, iproute2-6.19.0, libbpf 1.6.3
tcpdump version 4.99.6
libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
OpenSSL 3.5.5 27 Jan 2026
64-bit build, 64-bit time_t
Python 3.14.4
```

BAB 2: EKSPERIMEN 1 - PEMBANGUNAN & INSPEKSI TOPOLOGI JARINGAN

Pada eksperimen pertama, skrip pembangun jaringan network_lab.sh dieksekusi dengan argumen 'up' untuk membuat 3 network namespace, mengonfigurasi pasang pasangan virtual ethernet (veth-s0 ke veth-r0, dan veth-r1 ke veth-c0), menetapkan IP dual-stack, serta mengaktifkan IP forwarding pada namespace router.

2.1 Eksekusi Skrip & Pemeriksaan Alamat Antarmuka

Setiap namespace diverifikasi menggunakan perintah 'ip -br addr' untuk memastikan seluruh antarmuka virtual dalam status UP dengan penetapan IP yang presisi.

```

ridlo@ubuntu:~$ sudo bash network_lab.sh up
READY: iot-sensor <-> iot-router <-> iot-cloud
ridlo@ubuntu:~$ sudo ip -n iot-sensor -br addr
lo                UNKNOWN      127.0.0.1/8      ::1/128
s0@if2            UP                10.10.1.2/24     2001:db8:1::2/64 fe80::b81f:17ff:fe00:fdff/64
ridlo@ubuntu:~$ sudo ip -n iot-router -br addr
lo                UNKNOWN      127.0.0.1/8      ::1/128
r0@if2            UP                10.10.1.1/24     2001:db8:1::1/64 fe80::68c9:87ff:fe1b:c31e/64
r1@if2            UP                10.10.2.1/24     2001:db8:2::1/64 fe80::309a:8bff:fea5:39cd/64
ridlo@ubuntu:~$ sudo ip -n iot-cloud -br addr
lo                UNKNOWN      127.0.0.1/8      ::1/128
c0@if3            UP                10.10.2.2/24     2001:db8:2::2/64 fe80::3c14:6aff:fe94:8375/64

```

2.2 Pengujian Konektivitas End-to-End & Rute Next-Hop

Pengujian konektivitas dua arah dilakukan dari iot-sensor menuju iot-cloud untuk IPv4 (10.10.2.2) dan IPv6 (2001:db8:2::2). Selain itu, perintah 'ip route get' dieksekusi untuk membuktikan bahwa paket dikirim melalui gateway rute statis yang benar.

```

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -c 2 10.10.2.2
PING 10.10.2.2 (10.10.2.2) 56(84) bytes of data:
64 bytes from 10.10.2.2: icmp_seq=1 ttl=63 time=7.85 ms
64 bytes from 10.10.2.2: icmp_seq=2 ttl=63 time=0.112 ms

--- 10.10.2.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1041ms
rtt min/avg/max/mdev = 0.112/3.981/7.851/3.869 ms
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -6 -c 2 2001:db8:2::2
PING 2001:db8:2::2 (2001:db8:2::2) 56 data bytes
64 bytes from 2001:db8:2::2: icmp_seq=1 ttl=63 time=1.36 ms
64 bytes from 2001:db8:2::2: icmp_seq=2 ttl=63 time=0.073 ms

--- 2001:db8:2::2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.073/0.714/1.356/0.641 ms
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ip route get 10.10.2.2
10.10.2.2 via 10.10.1.1 dev s0 src 10.10.1.2 uid 0
cache
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ip -6 route get 2001:db8:2::2
2001:db8:2::2 from :: via 2001:db8:1::1 dev s0 src 2001:db8:1::2 metric 1024 pref medium

```

Analisis Checkpoint: Hasil 'route get 10.10.2.2' menunjukkan bahwa gateway next-hop adalah 10.10.1.1 melalui dev s0. Hal ini membuktikan bahwa tabel rute statis sensor berhasil mengarahkan seluruh lalu lintas luar subnet ke gateway router.

BAB 3: EKSPERIMEN 2 - PENGAMATAN ARP DAN IPV6 NEIGHBOR DISCOVERY

Eksperimen ini bertujuan untuk mengamati dan membandingkan mekanisme resolusi alamat link-layer (MAC address) antara IPv4 dan IPv6.

3.1 Resolusi Alamat IPv4 Menggunakan ARP

Sebelum mengirim paket data IPv4, host pengirim yang tidak memiliki pemetaan MAC next-hop akan menyiarkan paket ARP Request (Who has 10.10.1.1? Tell 10.10.1.2). Router membalas dengan ARP Reply (10.10.1.1 is at [MAC-r0]).

```

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor tcpdump -i s0 -nn -e -vv arp
tcpdump: listening on s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12:13:32.699724 ba:1f:17:00:fd:ff > ff:ff:ff:ff:ff:ff, ethertype ARP (0x8806), Length 42: Ethernet (len 6), IPv4 (len 4), Request who-has 10.10.1.1 tell 10.10.1.2, length 28
12:13:32.699860 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype ARP (0x8806), Length 42: Ethernet (len 6), IPv4 (len 4), Reply 10.10.1.1 is-at 6a:c9:87:1b:c3:1e, length 28
12:13:37.994981 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype ARP (0x8806), Length 42: Ethernet (len 6), IPv4 (len 4), Request who-has 10.10.1.2 tell 10.10.1.1, length 28
12:13:37.994995 ba:1f:17:00:fd:ff > 6a:c9:87:1b:c3:1e, ethertype ARP (0x8806), Length 42: Ethernet (len 6), IPv4 (len 4), Reply 10.10.1.2 is-at ba:1f:17:00:fd:ff, length 28

4 packets captured
4 packets received by filter
0 packets dropped by kernel

```

3.2 Resolusi Alamat IPv6 Menggunakan ICMPv6 Neighbor Discovery

IPv6 tidak menggunakan broadcast ARP, melainkan Neighbor Discovery Protocol (NDP) berbasis ICMPv6 multicast. Pengirim mengirimkan ICMPv6 Neighbor Solicitation (Type 135) ke alamat Solicited-Node Multicast, dan router membalas dengan ICMPv6 Neighbor Advertisement (Type 136).

```

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor tcpdump -i s0 -nn -e -vv icmp
tcpdump: listening on s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12:17:23.000000 ba:1f:17:00:fd:ff > ff:ff:ff:ff:ff:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) fdb8:bd1f:17ff:fdb8:fff > fdb2::1:ff00::1 [icmp sum ok] ICMPv6, neighbor solicitation, length 32, who has 2001:dbd::1:1
Source link-source option (1), length 8 (1): ba:1f:17:00:fd:ff
Destination link-source option (2), length 8 (1): ba:1f:17:00:fd:ff
12:17:23.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) 2001:dbd::1:1 > fdb8:bd1f:17ff:fdb8:fff [icmp sum ok] ICMPv6, neighbor advertisement, length 32, tgt is 2001:dbd::1:1, Flags [router, solicited, override]
Source link-source option (1), length 8 (1): 6a:c9:87:1b:c3:1e
Destination link-source option (2), length 8 (1): 6a:c9:87:1b:c3:1e
12:17:23.000000 6a:c9:87:1b:c3:1e > ff:ff:ff:ff:ff:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) 2001:dbd::1:2 > fdb2::1:ff00::1 [icmp sum ok] ICMPv6, neighbor solicitation, length 32, who has 2001:dbd::1:1
Source link-source option (1), length 8 (1): ba:1f:17:00:fd:ff
Destination link-source option (2), length 8 (1): ba:1f:17:00:fd:ff
12:17:23.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) 2001:dbd::1:1 > 2001:dbd::1:2 [icmp sum ok] ICMPv6, neighbor advertisement, length 32, tgt is 2001:dbd::1:1, Flags [router, solicited, override]
Source link-source option (1), length 8 (1): 6a:c9:87:1b:c3:1e
Destination link-source option (2), length 8 (1): 6a:c9:87:1b:c3:1e
12:17:23.000000 6a:c9:87:1b:c3:1e > 6a:c9:87:1b:c3:1e, ethertype IPv6 (0x86dd), Length 118: (fragLabel 0xa031, vln 64, next-header ICMPv6 (58), payload length 64) 2001:dbd::1:2 > 2001:dbd::2:2 [icmp sum ok] ICMPv6, echo request, seq 1
12:17:23.000000 6a:c9:87:1b:c3:1e > 6a:c9:87:1b:c3:1e, ethertype IPv6 (0x86dd), Length 118: (fragLabel 0xa032, vln 64, next-header ICMPv6 (58), payload length 64) 2001:dbd::2:2 > 2001:dbd::1:2 [icmp sum ok] ICMPv6, echo reply, seq 1
12:17:26.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) fdb8:bd1f:17ff:fdb8:fff > 2001:dbd::1:2 [icmp sum ok] ICMPv6, neighbor solicitation, length 32, who has 2001:dbd::1:2
Source link-source option (1), length 8 (1): 6a:c9:87:1b:c3:1e
Destination link-source option (2), length 8 (1): 6a:c9:87:1b:c3:1e
12:17:26.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) 2001:dbd::1:2 > fdb8:bd1f:17ff:fdb8:fff [icmp sum ok] ICMPv6, neighbor advertisement, length 32, tgt is 2001:dbd::1:2, Flags [solicited]
Source link-source option (1), length 8 (1): 6a:c9:87:1b:c3:1e
Destination link-source option (2), length 8 (1): 6a:c9:87:1b:c3:1e
12:17:30.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) fdb8:bd1f:17ff:fdb8:fff > fdb8:bd1f:17ff:fdb8:fff [icmp sum ok] ICMPv6, neighbor advertisement, length 32, tgt is fdb8:bd1f:17ff:fdb8:fff, Flags [solicited]
Source link-source option (1), length 8 (1): 6a:c9:87:1b:c3:1e
Destination link-source option (2), length 8 (1): 6a:c9:87:1b:c3:1e
12:17:30.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 80: (Vlan 255, next-header ICMPv6 (58), payload length 32) fdb8:bd1f:17ff:fdb8:fff > fdb8:bd1f:17ff:fdb8:fff [icmp sum ok] ICMPv6, neighbor solicitation, length 32, who has fdb8:bd1f:17ff:fdb8:fff
Source link-source option (1), length 8 (1): ba:1f:17:00:fd:ff
Destination link-source option (2), length 8 (1): ba:1f:17:00:fd:ff
12:17:30.000000 6a:c9:87:1b:c3:1e > ba:1f:17:00:fd:ff, ethertype IPv6 (0x86dd), Length 70: (Vlan 255, next-header ICMPv6 (58), payload length 24) fdb8:bd1f:17ff:fdb8:fff > fdb8:bd1f:17ff:fdb8:fff [icmp sum ok] ICMPv6, neighbor advertisement, length 24, tgt is fdb8:bd1f:17ff:fdb8:fff, Flags [router, solicited]
Source link-source option (1), length 8 (1): ba:1f:17:00:fd:ff
Destination link-source option (2), length 8 (1): ba:1f:17:00:fd:ff
12:17:30.000000 6a:c9:87:1b:c3:1e > 20:20:00:00:00:00, ethertype IPv6 (0x86dd), Length 70: (Vlan 255, next-header ICMPv6 (58), payload length 16) fdb8:bd1f:17ff:fdb8:fff > fdb2::2 [icmp sum ok] ICMPv6, router solicitation, length 16
Source link-source option (1), length 8 (1): ba:1f:17:00:fd:ff
Destination link-source option (2), length 8 (1): ba:1f:17:00:fd:ff
13 packets captured
13 packets received by filter
0 packets dropped by kernel

```

Analisis Checkpoint: Hasil penangkapan paket membuktikan bahwa alamat MAC tujuan yang teresolusi oleh sensor selalu alamat MAC antarmuka router (r0), BUKAN alamat MAC cloud (c0). Hal ini mengonfirmasi bahwa resolusi link-layer pada jaringan terkonfigurasi rute bersifat hop-by-hop.

BAB 4: EKSPERIMEN 3 - FORWARDING DAN DIAGNOSIS KEGAGALAN RUTE BALIK

Eksperimen ketiga menguji skenario kegagalan jaringan yang umum terjadi akibat kesalahan konfigurasi Hop Limit, rute balik, dan parameter penerusan kernel.

4.1 Uji Batas Hop Limit / TTL = 1

Saat sensor mengirim paket dengan TTL/Hop Limit = 1, router menerima paket tersebut tetapi mengurangi nilainya menjadi 0. Router menolak meneruskan paket dan mengembalikan respon ICMP Time Exceeded kembali ke pengirim.

```

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -t 1 -c 2 10.10.2.2
PING 10.10.2.2 (10.10.2.2) 56(84) bytes of data.
From 10.10.1.1 icmp_seq=1 Time to live exceeded
From 10.10.1.1 icmp_seq=2 Time to live exceeded

--- 10.10.2.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1006ms

```

4.2 Uji Simulasi Hilangnya Rute Balik (Missing Return Route)

Rute balik IPv4 pada namespace cloud dihapus (sudo ip -n iot-cloud route del 10.10.1.0/24). Saat sensor melakukan ping IPv4, paket request sampai di cloud, namun cloud tidak dapat mengirim paket Echo Reply kembali karena tidak menemukan rute ke 10.10.1.0/24.

```

ridlo@ubuntu:~$ sudo ip -n iot-cloud route del 10.10.1.0/24
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -c 2 10.10.2.2
PING 10.10.2.2 (10.10.2.2) 56(84) bytes of data.

--- 10.10.2.2 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1061ms

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -6 -c 2 2001:db8:2::2
PING 2001:db8:2::2 (2001:db8:2::2) 56 data bytes
64 bytes from 2001:db8:2::2: icmp_seq=1 ttl=63 time=0.116 ms
64 bytes from 2001:db8:2::2: icmp_seq=2 ttl=63 time=0.097 ms

--- 2001:db8:2::2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1044ms
rtt min/avg/max/mdev = 0.097/0.106/0.116/0.009 ms
ridlo@ubuntu:~$ sudo ip -n iot-cloud route add 10.10.1.0/24 via 10.10.2.1

```

4.3 Uji Matinya Fungsi IP Forwarding Router

Fungsi penyerutan paket pada router dimatikan dengan perintah 'sysctl -w net.ipv4.ip_forward=0'. Akibatnya, router menolak meneruskan seluruh lalu lintas antar-subnet.

```

ridlo@ubuntu:~$ sudo ip netns exec iot-router sysctl -w net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -c 2 10.10.2.2
PING 10.10.2.2 (10.10.2.2) 56(84) bytes of data.

--- 10.10.2.2 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1060ms

ridlo@ubuntu:~$ sudo ip netns exec iot-router sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -c 2 10.10.2.2
PING 10.10.2.2 (10.10.2.2) 56(84) bytes of data.
64 bytes from 10.10.2.2: icmp_seq=1 ttl=63 time=3.16 ms
64 bytes from 10.10.2.2: icmp_seq=2 ttl=63 time=0.442 ms

--- 10.10.2.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.442/1.799/3.156/1.357 ms

```

BAB 5: EKSPERIMEN 4 - PENGUJIAN CONSTRAINED PATH MTU DISCOVERY (PMTU)

Pada eksperimen ini, MTU antarmuka link kedua (r1 - c0) diturunkan menjadi 1280 bytes (batas MTU minimum IPv6).

5.1 Penanganan Paket Oversized IPv6 via ICMPv6 Packet Too Big

Sensor mengirim paket IPv6 dengan payload 1400 byte (total 1448 byte). Saat paket tiba di router, router tidak dapat meneruskannya ke link r1 karena MTU 1280. Karena IPv6 router tidak boleh memfragmentasi paket di tengah jalan, router membuang paket dan mengirimkan respon ICMPv6 Packet Too Big (Type 2 Code 0, MTU=1280) ke sensor.

```

ridlo@ubuntu:~$ sudo ip -n iot-router link set r1 mtu 1280
[sudo: authenticate] Password:
ridlo@ubuntu:~$ sudo ip -n iot-cloud link set c0 mtu 1280
ridlo@ubuntu:~$ sudo ip -n iot-router -br link
lo                UNKNOWN      00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
r0@if2            UP          6a:c9:87:1b:c3:1e <BROADCAST,MULTICAST,UP,LOWER_UP>
r1@if2            UP          32:9a:8b:a5:39:cd <BROADCAST,MULTICAST,UP,LOWER_UP>
ridlo@ubuntu:~$ sudo ip -n iot-cloud -br link
lo                UNKNOWN      00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
c0@if3            UP          3e:14:6a:94:83:75 <BROADCAST,MULTICAST,UP,LOWER_UP>
ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -6 -M do -s 1400 -c 2 2001:db8:2::2
PING 2001:db8:2::2 (2001:db8:2::2) 1400 data bytes
From 2001:db8:1::1 icmp_seq=1 Packet too big: mtu=1280
ping: sendmsg: Message too long

--- 2001:db8:2::2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1037ms

```

5.2 Pengujian Packet Fits Probe (Payload 1232 Bytes)

Ukuran payload disesuaikan menjadi 1232 byte ($1232 + 40 \text{ header IPv6} + 8 \text{ ICMPv6} = 1280 \text{ byte}$). Paket berhasil dikirim dan diterima tanpa kendala.

```

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor ping -6 -M do -s 1232 -c 2 2001:db8:2::2
PING 2001:db8:2::2 (2001:db8:2::2) 1232 data bytes
1240 bytes from 2001:db8:2::2: icmp_seq=1 ttl=63 time=0.141 ms
1240 bytes from 2001:db8:2::2: icmp_seq=2 ttl=63 time=0.075 ms

--- 2001:db8:2::2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1092ms
rtt min/avg/max/mdev = 0.075/0.108/0.141/0.033 ms

```

BAB 6: EKSPERIMEN 5 - KOMUNIKASI LAYANAN APLIKASI HTTP DI ATAS IPV6

Eksperimen akhir membuktikan bahwa infrastruktur jaringan IPv6 yang telah dibangun sanggup mendukung komunikasi tingkat aplikasi (HTTP di atas TCP).

6.1 Pelayanan Server HTTP IPv6 dan Eksekusi Klien Socket

Server HTTP Python dijalankan di namespace cloud terikat pada IPv6 address 2001:db8:2::2 port 8000. Klien socket Python di sensor meminta resource GET / HTTP/1.0 dan berhasil menerima respon HTTP 200 OK.

```

ridlo@ubuntu:~$ sudo ip netns exec iot-cloud python3 -m http.server 8000 --b
ind 2001:db8:2::2
Serving HTTP on 2001:db8:2::2 port 8000 (http://[2001:db8:2::2]:8000/) ...
2001:db8:1::2 - - [03/Oct/2026 12:53:45] "GET / HTTP/1.0" 200 -

ridlo@ubuntu:~$ sudo ip netns exec iot-sensor python3 -c "
import socket
s = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
s.connect(('2001:db8:2::2', 8000))
s.sendall(b'GET / HTTP/1.0\r\nHost: [2001:db8:2::2]\r\n\r\n')
response = s.recv(4096)
print(response.decode('utf-8'))
s.close()
"
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.4
Date: Sat, 03 Oct 2026 12:53:45 GMT
Content-type: text/html; charset=utf-8
Content-Length: 1947

```

BAB 7: TABULASI HASIL PENGUJIAN SKENARIO JARINGAN

Berikut adalah rekapitulasi lengkap hasil pengujian dari 9 skenario eksperimen yang dilaksanakan:

Skenario Pengujian	Expected Next Hop	Hasil Pengamatan	Bukti Paket / Log	Penjelasan Teknis
Baseline IPv4 Ping	10.10.1.1 (r0)	0% packet loss	ICMP Echo Reply dari 10.10.2.2	Rute statis dua arah dikonfigurasi dengan benar.
Baseline IPv6 Ping	2001:db8:1::1 (r0)	0% packet loss	ICMPv6 Echo Reply dari 2001:db8:2::2	Rute IPv6 /64 dan ND berjalan normal.
Missing Return Route IPv4	10.10.1.1 (r0)	100% packet loss	Request sampai di cloud, reply dibuang	Cloud tidak tahu rute balik ke 10.10.1.0/24.
Disabled IP Forwarding	None (Dropped at r0)	Destination Unreachable	Tidak ada paket keluar dari r0/r1	Router menolak meneruskan trafik antar-subnet.
Hop Limit / TTL = 1	10.10.1.1 (r0)	Time to Live Exceeded	ICMP Time Exceeded dari 10.10.1.1	Hop Limit habis saat mencapai router pertama.
IPv6 Oversized Probe (1400B)	2001:db8:1::1 (r0)	Packet Too Big	ICMPv6 Type 2 Code 0 (MTU=1280)	Router menolak paket melebihi MTU link r1 (1280).
IPv6 Fits Probe (1232B)	2001:db8:1::1 (r0)	0% packet loss	2 packets transmitted/received	Total paket (1232+40+8=1280B) pas dengan MTU.
IPv4 Oversized DF Probe	10.10.1.1 (r0)	Frag Needed (DF set)	ICMP Destination Unreachable / Frag Needed	Paket DF dibuang karena melebihi MTU 1280.
HTTP Server Over IPv6	2001:db8:1::1 (r0)	HTTP/1.0 200 OK	TCP SYN/ACK + HTTP Data 200 OK	Koneksi aplikasi TCP IPv6 berhasil end-to-end.

BAB 8: JAWABAN PERTANYAAN ANALISIS DAN EVALUASI

Pertanyaan 1: Pada topologi jaringan tanpa NAT, alamat manakah yang mengalami perubahan pada setiap hop yang dirutekan: alamat IP asal/tujuan, ataukah alamat link-layer (MAC) asal/tujuan?

Jawaban Analitis:

1. Alamat IP Asal dan Tujuan (Source/Destination IP): Tetap KONSTAN (TIDAK BERUBAH) dari pengirim awal hingga penerima akhir. Pada eksperimen ini, IP asal tetap 10.10.1.2 (IPv4) / 2001:db8:1::2 (IPv6) dan IP tujuan tetap 10.10.2.2 (IPv4) / 2001:db8:2::2 (IPv6). Karena tidak ada perangkat NAT di sepanjang jalur, header IP dipertahankan secara end-to-end.
2. Alamat Link-Layer / MAC Address: BERUBAH di setiap hop yang dirutekan. Saat paket meninggalkan sensor menuju router, Frame Ethernet menggunakan MAC Asal = MAC antarmuka s0 dan MAC Tujuan = MAC antarmuka r0. Ketika router meneruskan paket dari antarmuka r1

menuju cloud, router membungkus kembali paket IP ke dalam Frame Ethernet baru dengan MAC Asal = MAC antarmuka r1 dan MAC Tujuan = MAC antarmuka c0. Hal ini membuktikan bahwa pengalaman MAC hanya berlaku lokal pada satu segmen link (hop-by-hop).

Pertanyaan 2: Mengapa perangkat cloud endpoint harus memiliki dan mengetahui rute balik (return route) menuju ke subnet sensor?

Jawaban Analitis:

Komunikasi jaringan IP bersifat dua arah (bidirectional). Meskipun rute maju (forward route) dari sensor ke cloud dikonfigurasi dengan sempurna sehingga paket data (seperti ICMP Echo Request atau TCP SYN) berhasil diterima oleh cloud, cloud harus membalas paket tersebut (mengirim ICMP Echo Reply atau TCP SYN-ACK).

Saat akan membalas, stack IP cloud memeriksa tabel rutenya untuk menemukan jalur menuju alamat IP tujuan 10.10.1.2 (subnet sensor 10.10.1.0/24). Jika rute balik ini hilang (seperti yang dibuktikan pada Eksperimen 3), cloud tidak tahu ke mana harus mengarahkan paket balasan tersebut. Akibatnya, paket balasan dibuang di sisi cloud, dan pengirim (sensor) mengalami kegagalan koneksi (timeout).

Pertanyaan 3: Mengapa tindakan memblokir seluruh lalu lintas protokol ICMPv6 dapat merusak fungsi operasional jaringan IPv6 melebihi sekadar perintah ping?

Jawaban Analitis:

Tidak seperti pada IPv4 di mana ICMP bersifat opsional (hanya untuk diagnostik), ICMPv6 adalah komponen inti dan integral dari arsitektur operasional IPv6. Memblokir seluruh ICMPv6 akan merusak:

1. **Neighbor Discovery Protocol (NDP):** IPv6 tidak memiliki ARP. Resolusi MAC address dilakukan menggunakan ICMPv6 Neighbor Solicitation (Type 135) dan Neighbor Advertisement (Type 136). Memblokir ICMPv6 membuat perangkat tidak bisa menemukan MAC address tetangganya, sehingga komunikasi terhenti total.
2. **Path MTU Discovery (PMTU):** IPv6 router tidak melakukan fragmentasi paket. Jika paket melebihi MTU link, router mengirimkan pesan ICMPv6 Packet Too Big (Type 2). Jika pesan ini diblokir, pengirim tidak pernah tahu bahwa paketnya terlalu besar (menciptakan fenomena PMTU Black Hole).
3. **Autokonfigurasi Alamat (SLAAC):** Router Discovery menggunakan ICMPv6 Router Solicitation (Type 137) dan Router Advertisement (Type 138) untuk membagikan prefix IPv6 secara otomatis.

Pertanyaan 4: Sebuah payload UDP menggunakan header UDP berukuran 8 byte. Berapakah ukuran payload UDP maksimum yang dapat muat di dalam satu paket IPv6 berukuran 1280 byte tanpa adanya extension header?

Jawaban Analitis:

Kalkulasi matematis batas payload UDP di atas IPv6 MTU 1280 bytes:

- **Total Batas MTU IPv6 Link:** 1280 bytes
- **Ukuran Header Standar IPv6:** 40 bytes (fixed header)

- **Ukuran Header UDP: 8 bytes**

Rumus Kalkulasi Payload UDP Maksimum:

$$\text{Payload UDP Maksimum} = \text{MTU} - \text{Header IPv6} - \text{Header UDP}$$

$$\text{Payload UDP Maksimum} = 1280 - 40 - 8 = 1232 \text{ bytes}$$

Kesimpulan: Ukuran payload data aplikasi UDP maksimum yang dapat dikirimkan dalam satu paket IPv6 tanpa memicu fragmentasi pada MTU 1280 adalah tepat 1232 bytes (seperti yang dibuktikan pada Eksperimen 4).

Pertanyaan 5: Jelaskan perbedaan antara Ethernet MTU, ukuran frame radio nirkabel, dan fragmentasi pada lapisan adaptasi 6LoWPAN!

Jawaban Analitis:

1. **Ethernet MTU:** Batas maksimum ukuran paket IP yang dapat ditransmisikan melalui media kabel Ethernet. Standar Ethernet MTU adalah 1500 bytes, sedangkan spesifikasi IPv6 mensyaratkan link MTU minimum sebesar 1280 bytes.
2. **Ukuran Frame Radio Nirkabel (Radio Frame Size):** Batas fisik muatan pada media nirkabel berdaya rendah. Sebagai contoh, standar IEEE 802.15.4 (digunakan oleh Zigbee/Thread) memiliki Maximum Physical Layer Packet Size (PSDU) hanya sebesar 127 bytes.
3. **Fragmentasi Lapisan Adaptasi 6LoWPAN:** Lapisan perantara (adaptation layer) yang berada di antara Data Link Layer (802.15.4) dan Network Layer (IPv6). Karena IPv6 mewajibkan MTU minimum 1280 bytes sementara radio 802.15.4 hanya sanggup membawa ~127 bytes, 6LoWPAN bertugas mengompresi header IPv6/UDP dan memotong (fragmentasi) paket IPv6 1280 bytes menjadi beberapa frame radio kecil di sisi pengirim, lalu merakitnya kembali sebelum diserahkan ke stack IPv6 penerima.

BAB 9: PANDUAN TROUBLESHOOTING, PEMBERSIHAN, DAN KESIMPULAN

9.1 Pembersihan Topologi Jaringan

Setelah seluruh pengujian selesai, lingkungan kerja dibersihkan dengan menghapus namespace dan pasangan veth menggunakan skrip `network_lab.sh` argumen 'down'.

```
ridlo@ubuntu:~$ sudo bash network_lab.sh down
ridlo@ubuntu:~$ sudo ip netns list
ridlo@ubuntu:~$ |
```

9.2 Kesimpulan Praktikum

Praktikum Lapisan Internet ini berhasil membuktikan secara eksperimental mekanisme operasional IPv4 dan IPv6 dalam jaringan IoT terisolasi. Penggunaan dual-stack IPv6 memperlihatkan efisiensi resolusi alamat berbasis NDP multicast dibandingkan broadcast ARP IPv4. Selain itu, pengujian PMTU

menegaskan pentingnya penyesuaian ukuran payload aplikasi terhadap batasan MTU link untuk mencegah kehilangan paket akibat fragmentasi yang tertolak.

BAB 10: LAMPIRAN KODE SUMBER

10.1 Skrip Shell Pembangun Jaringan (network_lab.sh)

```
#!/usr/bin/env bash
# network_lab.sh - Script Penyetelan Topologi Internet Layer IoT
set -euo pipefail

cmd="${1:-up}"

if [ "$cmd" = "up" ]; then
    echo "Membangun topologi network namespaces..."
    sudo ip netns add iot-sensor
    sudo ip netns add iot-router
    sudo ip netns add iot-cloud

    # Veth pair Sensor - Router
    sudo ip link add veth-s0 type veth peer name veth-r0
    sudo ip link set veth-s0 netns iot-sensor dev s0
    sudo ip link set veth-r0 netns iot-router dev r0

    # Veth pair Router - Cloud
    sudo ip link add veth-r1 type veth peer name veth-c0
    sudo ip link set veth-r1 netns iot-router dev r1
    sudo ip link set veth-c0 netns iot-cloud dev c0

    # IP Setup Sensor
    sudo ip -n iot-sensor link set s0 up
    sudo ip -n iot-sensor addr add 10.10.1.2/24 dev s0
    sudo ip -n iot-sensor addr add 2001:db8:1::2/64 dev s0
    sudo ip -n iot-sensor route add default via 10.10.1.1 dev s0
    sudo ip -n iot-sensor -6 route add default via 2001:db8:1::1 dev s0

    # IP Setup Router
    sudo ip -n iot-router link set r0 up
    sudo ip -n iot-router link set r1 up
    sudo ip -n iot-router addr add 10.10.1.1/24 dev r0
    sudo ip -n iot-router addr add 2001:db8:1::1/64 dev r0
    sudo ip -n iot-router addr add 10.10.2.1/24 dev r1
    sudo ip -n iot-router addr add 2001:db8:2::1/64 dev r1
    sudo ip netns exec iot-router sysctl -q -w net.ipv4.ip_forward=1
    sudo ip netns exec iot-router sysctl -q -w net.ipv6.conf.all.forwarding=1

    # IP Setup Cloud
    sudo ip -n iot-cloud link set c0 up
    sudo ip -n iot-cloud addr add 10.10.2.2/24 dev c0
    sudo ip -n iot-cloud addr add 2001:db8:2::2/64 dev c0
    sudo ip -n iot-cloud route add default via 10.10.2.1 dev c0
    sudo ip -n iot-cloud -6 route add default via 2001:db8:2::1 dev c0

    echo "Topologi berhasil dibangun!"
elif [ "$cmd" = "down" ]; then
    echo "Menghapus topologi..."
```

```

    sudo ip netns del iot-sensor || true
    sudo ip netns del iot-router || true
    sudo ip netns del iot-cloud || true
    echo "Pembersihan selesai!"
fi

```

10.2 Skrip Python Klien Socket TCP IPv6

```

# ipv6_http_client.py - Klien HTTP Socket TCP IPv6
import socket

SERVER_IPV6 = "2001:db8:2::2"
PORT = 8000

def main():
    print(f"Connecting to http://[{SERVER_IPV6}]:{PORT}/...")
    # Socket AF_INET6 for IPv6 TCP
    with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as s:
        s.connect((SERVER_IPV6, PORT))
        request = "GET / HTTP/1.0\r\nHost: [2001:db8:2::2]\r\n\r\n"
        s.sendall(request.encode('utf-8'))

        response = b""
        while True:
            data = s.recv(1024)
            if not data:
                break
            response += data

        print("Response received from IPv6 HTTP Server:")
        print(response.decode('utf-8', errors='ignore'))

if __name__ == "__main__":
    main()

```